Pedestrian Collision Severity in London: Does Proximity to Greenspace Access Points Improve Prediction of Severe Pedestrian Injury Outcomes?

Preparation

- GitHub Repository: Click to view link
- Word Count (narrative text only, within 10% leeway of 1650 word limit from 1500 +10%): 1647 words
- Runtime: 157 seconds (*x86_64 processor, 20 logical CPU cores, 15.43 GB RAM*)
- Coding environment: Python 3.12.11 on Linux 6.6.87.2-microsoft-standard-WSL2
- License: this notebook is made available under the Creative Commons Attribution license (or other license that you like).
- Additional library *[libraries not included in SDS Docker or not used in this module]*:
  - **watermark**: A Jupyter Notebook extension for printing timestamps, version numbers, and hardware information.
  - **shap**: used for model interpretation
- Declaration: LLM has been used for fixing coding mistakes and tidy writings.

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and Discussion
6. Conclusion
7. References

1. Introduction

[go back to the top]

Pedestrian road safety is an urban policy and health challenge. Road collisions interplay between human behaviour, vehicle characteristics, and infrastructure conditions, with pedestrians among the most vulnerable road users, facing disproportionately severe injury outcomes relative to other road users (Pour-Rouholamin and Zhou, 2016; Hossain et al., 2026). The World Health Organization (2023) reported that over half of global road traffic fatalities involve vulnerable road users, underscoring the urgency of understanding the factors shaping injury severity. Walking is also a fundamental mode of urban transport, delivering benefits for health, community, and environmental sustainability (Mayor of London and Transport for London, n.d.).

In London, road safety and urban greening are both central policy objectives. The Vision Zero targets the elimination of all deaths and serious injuries from road collisions by 2041, with safer streets, speeds, and junction design (Mayor of London and Transport for London, n.d.; Transport for London, n.d.). Simultaneously, the London Plan 2021 recognises green infrastructure as a facilitator of walking and urban health, with urban greening positioned as an incentive to make walking more attractive (Mayor of London, 2021). Despite policy commitments, while existing literature identifies demographics, vehicle, road, lighting and junction characteristics as primary severity determinants, greenspace accessibility as an explanatory factor and in the London context remains underexplored (Clifton, Burnier and Akar, 2009; Malin, Silla and Mladenović, 2020; Elalouf, Birfir and Rosenbloom, 2023; Al Hasanat et al., 2025).

This analysis addresses that gap by examining London pedestrian casualty data for 2024, identifying variables associated with fatal/ serious injury outcomes and testing whether proximity to urban greenspace access points improves predictive performance after accounting for casualty, temporal, road, and vehicle characteristics. It is important to note that this analysis models injury severity conditional on a pedestrian casualty already having been recorded, and findings should therefore be interpreted as the probability of a recorded casualty being fatal/ serious rather than as an overall measure of collision risk.

2. Research questions

[go back to the top]

RQ1: Which casualty, temporal, road and vehicle factors are most important for predicting fatal or serious pedestrian injury outcomes in London in 2024?

RQ2: Does proximity to urban greenspace access points improve the prediction of fatal or serious pedestrian injury outcomes after accounting for casualty, road, vehicle, temporal and

spatial context?

3. Data

[go back to the top]

This analysis uses road collision, casualty, and vehicle datasets filtered for London and 2024. Datasets were merged at the casualty level to model pedestrian injury severity. Greenspace proximity was calculated as distance from each collision location to the nearest greenspace access point using OS Open Greenspace data.

Data preprocessing included handling ambiguous codes, recoding missing values, constructing numeric, categorical and temporal variables. The final dataset contains 4,445 pedestrian casualties, with 26.9% classified as fatal/ serious.

The analysis is conducted at casualty level rather than collision level, therefore each row represents one pedestrian casualty. Only 127 collisions involved multiple pedestrian casualties, with a maximum of four, creating minor dependence issues.

3.1. Load Packages

In [1]:
```
# Packages to install first prior to loading packages
# %pip install shap
```

In [2]:
```
# Data wrangling
import numpy as np
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
from scipy.stats import spearmanr
import warnings
warnings.filterwarnings('ignore')

# Pre-processing
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Train test split and cross-validation
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV

# Modelling
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Model Evaluation
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, matthews_corrcoef,
```

```
    classification_report, confusion_matrix,
    ConfusionMatrixDisplay, PrecisionRecallDisplay
)

# Interpretation
import shap

# Plot
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
```

3.2. Data Loading

In [3]:
```
# Load Collision 2024
collision = pd.read_csv("https://raw.githubusercontent.com/christychoicc/london-ped

# Load Casualty 2024
casualty = pd.read_csv("https://raw.githubusercontent.com/christychoicc/london-pede

# Load Vehicle 2024
vehicle = pd.read_csv("https://raw.githubusercontent.com/christychoicc/london-pedes

# Load Green Space data
greenspace = gpd.read_file("https://raw.githubusercontent.com/christychoicc/london-

# Load London MOSA boundary data
london_msoa = gpd.read_file("https://raw.githubusercontent.com/christychoicc/london

# Standardise column names
collision.columns = collision.columns.str.strip().str.lower()
casualty.columns = casualty.columns.str.strip().str.lower()
vehicle.columns = vehicle.columns.str.strip().str.lower()
greenspace.columns = greenspace.columns.str.strip().str.lower()
```

3.3. Data Pre-processing

In [4]:
```
# Data Cleaning

# Filter to London collisions only
collision_london = collision[
    collision['local_authority_ons_district'].astype(str).str.startswith('E09')
].copy()

# print("London collisions shape:", collision_london.shape)
# print(collision_london['local_authority_ons_district'].nunique())
```

In [5]:
```
# Filter Greenspace data to London only
# Using london_msoa dataset to set london boundary
london_boundary = london_msoa.dissolve()
# Ensure consistent CRS
greenspace = greenspace.to_crs(london_boundary.crs)
# Clip greenspace to the exact London boundary
```

```
greenspace_london = gpd.clip(greenspace, london_boundary)


# Transform geometry column's point(x y z) into normal point(x y) only
greenspace_london['geometry'] = greenspace_london['geometry'].apply(
    lambda g: Point(g.x, g.y)  # drops Z, keeps X and Y only
)

# Create collision points for calculating nearest greenspace distance
collision_points = gpd.GeoDataFrame(
    # remove duplicated collision_index
    collision_london[['collision_index', 'longitude', 'latitude']].dropna().drop_du
    # convert lat and long to geometry
    geometry=gpd.points_from_xy(
        collision_london[['collision_index', 'longitude', 'latitude']].dropna().dro
        collision_london[['collision_index', 'longitude', 'latitude']].dropna().dro
    ),
    crs='EPSG:4326'
).to_crs('EPSG:27700')

# Nearest greenspace access point
collision_green_nearest = gpd.sjoin_nearest(
    collision_points,
    greenspace_london[['geometry']],
    how='left',
    distance_col='dist_to_greenspace_m'
)[['collision_index', 'dist_to_greenspace_m']].drop_duplicates('collision_index')

# Merge newly computed collisioin to nearest greenspace access point with london co
collision_london = collision_london.merge(
    collision_green_nearest,
    on='collision_index',
    how='left'
)
```

In [6]:
```
# Filter to Pedestrian casualties only
# 3 = Pedestrians
casualty_ped = casualty[casualty['casualty_class'] == 3].copy()
```

In [7]:
```
# Filter for London Pedestrian casualties only
casualty_ped_london = casualty_ped[
    casualty_ped['collision_index'].isin(collision_london['collision_index'])
].copy()
```

In [8]:
```
# Check how many collision have more than one pedestrian casualty
peds_per_collision = casualty_ped_london.groupby('collision_index').size()

print("Number of pedestrian-collision rows:", len(casualty_ped_london))
print("Number of unique collisions involving pedestrians:", casualty_ped_london['co
print("Number of collisions with >1 pedestrian casualty:", (peds_per_collision > 1)
print("Maximum pedestrians in a single collision:", peds_per_collision.max())
```

```
Number of pedestrian-collision rows: 4445
Number of unique collisions involving pedestrians: 4299
Number of collisions with >1 pedestrian casualty: 127
Maximum pedestrians in a single collision: 4
```

In [9]:
```python
# Data Merge/ Join

# Merge casualty, collision, and vehicle data
df_merge = (
    # Merge pedestrian casualties
    casualty_ped_london
    .merge(
        # with London collisions
        collision_london.drop(columns=['collision_year', 'collision_ref_no'], error
        on='collision_index',
        how='left'
    )
    .merge(
        # with vehicles
        vehicle.drop(columns=['collision_year', 'collision_ref_no'], errors='ignore
        on=['collision_index', 'vehicle_reference'],
        how='left'
    )
)
```

3.4. Data Wrangling

3.4.1. Data Ambiguity

In [10]:
```python
# # Check missing and invalid temporal variable values
# datetime_check = df_merge[['collision_index', 'date', 'time']].copy()

# date_parsed_check = pd.to_datetime(
#     datetime_check['date'],
#     errors='coerce',
#     dayfirst=True
# )

# time_parsed_check = pd.to_datetime(
#     datetime_check['time'],
#     format='%H:%M',
#     errors='coerce'
# )

# invalid_dates = datetime_check[
#     datetime_check['date'].notna() & date_parsed_check.isna()
# ]

# invalid_times = datetime_check[
#     datetime_check['time'].notna() & time_parsed_check.isna()
# ]

# temporal_missing_invalid_summary = pd.DataFrame({
#     'variable': ['date', 'time'],
```

```
#     'missing_count': [
#         datetime_check['date'].isna().sum(),
#         datetime_check['time'].isna().sum()
#     ],
#     'invalid_count': [
#         invalid_dates.shape[0],
#         invalid_times.shape[0]
#     ]
# })

# temporal_missing_invalid_summary
```

In [11]:

```
# Check ambiguity of potentially needed variables
ambiguous_codes = {
    # casualty (pedestrian data)
    'sex_of_casualty':          [-1, 9],
    'age_of_casualty':          [-1],
    'pedestrian_location':      [-1, 10],
    'pedestrian_movement':      [-1, 9],
    'pedestrian_road_maintenance_worker': [-1, 2], # also a type of pedestrian so i
    'casualty_imd_decile':      [-1],

    # collision / road environment
    'speed_limit':              [-1, 99],
    'first_road_class':         [-1],
    'road_type':                [-1, 9],
    'junction_detail':          [-1, 99],
    'junction_control':         [-1, 9],
    'pedestrian_crossing':      [-1, 99],
    'light_conditions':         [-1],
    'weather_conditions':       [-1, 9],
    'road_surface_conditions':  [-1, 9],
    'urban_or_rural_area':      [-1],
    'trunk_road_flag':          [-1],

    # vehicle / driver
    'vehicle_type':             [-1, 99],
    'vehicle_manoeuvre':        [-1, 99],
    'junction_location':        [-1, 9],
    'sex_of_driver':            [-1, 3],
    'age_of_driver':            [-1],
    'driver_imd_decile':        [-1],
    'engine_capacity_cc':       [-1],
    'age_of_vehicle':           [-1]
}

# Print a summary table of unknown missing values
inspection_rows = []
for col, codes in ambiguous_codes.items():
    if col not in df_merge.columns:
        continue

    total= len(df_merge)
    actual_nan= df_merge[col].isnull().sum()
    ambig_count= df_merge[col].isin(codes).sum()
    total_missing= actual_nan + ambig_count
```

```
    pct= total_missing / total * 100
    inspection_rows.append({
        'variable': col,
        'total_rows': total,
        'actual_NaN': actual_nan,
        'ambiguous_codes': str(codes),
        'ambiguous_count': ambig_count,
        'total_missing': total_missing,
        'pct_missing': round(pct, 1)
    })

inspection_df = pd.DataFrame(inspection_rows).sort_values('pct_missing', ascending=
print("Missing / Unknown Data Inspection:")
print(inspection_df.to_string(index=False))
```

```
Missing / Unknown Data Inspection:
                        variable  total_rows  actual_NaN ambiguous_codes  ambiguou
s_count  total_missing  pct_missing
                engine_capacity_cc        4445           0            [-1]
2134           2134         48.0
                    age_of_vehicle        4445           0            [-1]
1904           1904         42.8
                  driver_imd_decile        4445           0            [-1]
1811           1811         40.7
                     sex_of_driver        4445           0         [-1, 3]
1539           1539         34.6
                     age_of_driver        4445           0            [-1]
1525           1525         34.3
               pedestrian_movement        4445           0         [-1, 9]
1523           1523         34.3
                  junction_control        4445           0         [-1, 9]
1315           1315         29.6
                  vehicle_manoeuvre        4445           0        [-1, 99]
1236           1236         27.8
                  junction_location        4445           0         [-1, 9]
995            995          22.4
               casualty_imd_decile        4445           0            [-1]
608            608          13.7
               pedestrian_location        4445           0        [-1, 10]
432            432           9.7
                   junction_detail        4445           0        [-1, 99]
363            363           8.2
                         road_type        4445           0         [-1, 9]
359            359           8.1
               pedestrian_crossing        4445           0        [-1, 99]
320            320           7.2
                weather_conditions        4445           0         [-1, 9]
308            308           6.9
          road_surface_conditions        4445           0         [-1, 9]
214            214           4.8
                   age_of_casualty        4445           0            [-1]
135            135           3.0
                      vehicle_type        4445           0        [-1, 99]
58             58            1.3
pedestrian_road_maintenance_worker        4445           0         [-1, 2]
56             56            1.3
                   sex_of_casualty        4445           0         [-1, 9]
42             42            0.9
                       speed_limit        4445           0        [-1, 99]
0              0             0.0
                  first_road_class        4445           0            [-1]
0              0             0.0
                   trunk_road_flag        4445           0            [-1]
0              0             0.0
                  light_conditions        4445           0            [-1]
0              0             0.0
               urban_or_rural_area        4445           0            [-1]
0              0             0.0
```

Variable	Code	Interpretation	Action
pedestrian_movement	-1, 9	Missing / unknown or other	Recode to `NaN`, impute with categorical `Missing`
junction_control	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
vehicle_manoeuvre	-1, 99	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
junction_location	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
casualty_imd_decile	-1	Missing	Recode to `NaN`, impute with categorical `Missing`
pedestrian_location	-1, 10	Missing / unknown or other	Recode to `NaN`, impute with categorical `Missing`
junction_detail	-1, 99	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
road_type	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
pedestrian_crossing	-1, 99	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
weather_conditions	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`, keep `8 = Other`
road_surface_conditions	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
age_of_casualty	-1	Missing	Recode to `NaN`, impute with median
vehicle_type	-1, 99	Missing / unknown vehicle type	Recode to `NaN`, impute with categorical `Missing`, keep `90 = Other vehicle`
pedestrian_road_maintenance_worker	-1, 2	Missing / not known	Recode to `NaN`, impute with categorical `Missing`
sex_of_casualty	-1, 9	Missing / unknown	Recode to `NaN`, impute with categorical `Missing`
speed_limit	-1, 99	Missing / unknown	No action needed for no ambiguous values found in dataset
first_road_class	-1	Missing / unknown	No action needed for no ambiguous values found in dataset

Variable	Code	Interpretation	Action
light_conditions	-1	Missing	No action needed for no ambiguous values found in dataset
urban_or_rural_area	-1	Missing	No action needed for no ambiguous values found in dataset
trunk_road_flag	-1	Missing	No action needed for no ambiguous values found in dataset
driver_imd_decile	-1	Missing	Exclude from modelling due to high levels of missing data
sex_of_driver	-1, 3	Missing / not known	Exclude from modelling due to high levels of missing data
age_of_driver	-1	Missing	Exclude from modelling due to high levels of missing data
engine_capacity_cc	-1	Missing	Exclude from modelling due to high levels of missing data
age_of_vehicle	-1	Missing	Exclude from modelling due to high levels of missing data

In [12]:
```
# Variable that needs recoding
recode_nan = {
    # collision
    'junction_control': [-1, 9],
    'junction_detail': [-1, 99],
    'road_type': [-1, 9],
    'pedestrian_crossing': [-1, 99],
    'weather_conditions': [-1, 9],
    'road_surface_conditions': [-1, 9],

    # casualty
    'pedestrian_movement': [-1, 9],
    'casualty_imd_decile': [-1],
    'pedestrian_location': [-1, 10],
    'age_of_casualty': [-1],
    'pedestrian_road_maintenance_worker': [-1, 2],
    'sex_of_casualty': [-1, 9],

    # vehicle
    'vehicle_manoeuvre': [-1, 99],
    'junction_location': [-1, 9],
```

```
    'vehicle_type': [-1, 99] # keep 90 = Others
}
```

In [13]:
```
# Set recode function
def apply_recode(df, recode_dict):
    df = df.copy()
    for col, bad_codes in recode_dict.items():
        if col in df.columns:
            df[col] = df[col].replace(bad_codes, np.nan)
    return df
```

In [14]:
```
# Apply recode functions to the merged dataset
df_merge_recode = apply_recode(df_merge, recode_nan)
```

3.4.2. Variables Wrangling

In [15]:
```
# Extract temporal features from collision date and time

# Convert time to hour for easier interpretation
df_merge_recode['time'] = pd.to_datetime(
    df_merge_recode['time'], format='%H:%M', errors='coerce'
)
df_merge_recode['hour'] = df_merge_recode['time'].dt.hour

# Define time periods categories
def assign_time_period(hour):
    if pd.isna(hour):
        return np.nan
    elif 7 <= hour <= 9:
        return 'Morning Peak'
    elif 10 <= hour <= 15:
        return 'Daytime'
    elif 16 <= hour <= 18:
        return 'Evening Peak'
    elif 19 <= hour <= 22:
        return 'Evening'
    else:
        return 'Night' # midnight to early morning

df_merge_recode['time_period'] = df_merge_recode['hour'].apply(assign_time_period)

# Create weekday and weekend
df_merge_recode['date'] = pd.to_datetime(
    df_merge_recode['date'],
    errors='coerce',
    dayfirst=True
)
df_merge_recode['day_of_week'] = df_merge_recode['date'].dt.day_name()
df_merge_recode['day_type'] = df_merge_recode['date'].dt.dayofweek.apply(
    lambda x: 'Weekend' if x >= 5 else 'Weekday'
)

# # Check distributions
# print("Time period distribution:")
# print(df_merge_recode['time_period'].value_counts())
```

```
# print("\nDay type distribution:")
# print(df_merge_recode['day_type'].value_counts())
# print("\nSeverity rate by time period:")
# print(df_merge_recode.groupby('time_period')['casualty_severity']
#       .apply(lambda x: (x.isin([1,2])).mean()).round(3).sort_values(ascending=Fal
```

In [16]:
```
# Prepare data variables needed for exploratory analysis
# Assign each pedestrian casualty to MSOA using collision coordinates

# Project MSOA boundary to British National Grid
london_msoa = london_msoa.to_crs("EPSG:27700")
london_msoa["geometry"] = london_msoa["geometry"].buffer(0)

# Create casualty-level point geometry from collision longitude/latitude
df_points = gpd.GeoDataFrame(
    df_merge_recode.dropna(subset=["longitude", "latitude"]).copy(),
    geometry=gpd.points_from_xy(
        df_merge_recode.dropna(subset=["longitude", "latitude"])["longitude"],
        df_merge_recode.dropna(subset=["longitude", "latitude"])["latitude"]
    ),
    crs="EPSG:4326"
).to_crs("EPSG:27700")

# Spatial join casualty points to MSOA polygons
df_points_msoa = gpd.sjoin(
    df_points,
    london_msoa[["MSOA11CD", "MSOA11NM", "geometry"]],
    how="left",
    predicate="within"
)

# Merge MSOA codes back to the main dataframe
df_merge_recode = df_merge_recode.merge(
    df_points_msoa[["collision_index", "vehicle_reference", "casualty_reference", "
    on=["collision_index", "vehicle_reference", "casualty_reference"],
    how="left"
)
```

In [17]:
```
# # Check merged columns
# print("\n Merged Dataframe Columns:")
# print(df_merge_recode.columns.tolist())
```

In [18]:
```
# Data Filtering

# Select needed columns
selected_columns = [
    # To be used to create our target binary variable
    'casualty_severity',

    # casualty, pedestrian variables
    'sex_of_casualty',
    'age_of_casualty',
    'pedestrian_location',
    'pedestrian_movement',
    'pedestrian_road_maintenance_worker',
```

```
    'casualty_imd_decile',

    # collision, road and environment variables
    'speed_limit',
    'first_road_class',
    'road_type',
    'junction_detail',
    'junction_control',
    'pedestrian_crossing',
    'light_conditions',
    'weather_conditions',
    'road_surface_conditions',
    'urban_or_rural_area',
    'trunk_road_flag',

    # vehicle, driver variables
    'vehicle_type',
    'vehicle_manoeuvre',
    'junction_location',
    # 'sex_of_driver', # Excluded for high levels missing data
    # 'age_of_driver', # Excluded for high levels missing data
    # 'age_of_vehicle', # Excluded for high levels missing data
    # 'driver_imd_decile', # Excluded for high levels missing data

    # spatial contextual variable
    'dist_to_greenspace_m',

    # temporal variables
    'time_period',
    'day_of_week',
    'day_type',

    # exploratory mapping purpose
    'MSOA11CD',
    'MSOA11NM'
]

selected_columns = [c for c in selected_columns if c in df_merge_recode.columns]

df_clean = df_merge_recode[selected_columns].copy()
```

In [19]:

```
# Data Type

# Define features by data type and datasets groups

casualty_numeric = ['age_of_casualty']
casualty_categorical = [
    'sex_of_casualty', 'pedestrian_location', 'pedestrian_movement',
    'pedestrian_road_maintenance_worker', 'casualty_imd_decile']

# No numeric features for raod
road_categorical = [
    'speed_limit', 'first_road_class', 'road_type', 'junction_detail',
    'junction_control', 'pedestrian_crossing', 'light_conditions',
    'weather_conditions', 'road_surface_conditions',
    'urban_or_rural_area', 'trunk_road_flag'
```

```
]

# No numeric features for vehicle
vehicle_categorical = ['vehicle_type', 'vehicle_manoeuvre', 'junction_location']

# No numeric features for greenspace
greenspace_numeric = ['dist_to_greenspace_m']

# No numeric features for temporal variables
temporal_categorical = ['time_period', 'day_of_week', 'day_type']

# All features in numeric and categorical
numeric_features = casualty_numeric + greenspace_numeric
categorical_features = casualty_categorical + road_categorical + vehicle_categorica

# Note*
# severity_levels are targets so not included here
# casualty_severity used to create severity levels so not included here
# spatial variables not included here
```

In [20]:
```
# Assign categroical features as objects

# Numeric columns
# Keep values as numeric
for col in numeric_features:
    if col in df_clean.columns:
        df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')

# Categorical columns
# Convert all non-missing values to strings --> keep missing as NaN
for col in categorical_features:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].apply(lambda x: np.nan if pd.isna(x) else str

# # Check numeric features are numeric
# print("\nNumeric Features:")
# print(df_clean[numeric_features].dtypes)

# # Check categorical features are strings
# print("\nCategorical Features:")
# print(df_clean[categorical_features].dtypes)

# # Check NaN is preserved correctly (not converted to string "nan")
# print("\nNaNs:")
# print(df_clean[categorical_features].isnull().sum()[df_clean[categorical_features
```

3.4.3. Create Target Variable

Grouping fatal and serious injuries addressed class imbalance and aligned with policy focus on severe outcomes. The binary target variable `severity_levels`:

- `1` = fatal or serious pedestrian injury
- `0` = slight pedestrian injury

In [21]:
```
# Check initial distribution of casualty severity categories
print("Casualty severity data distribution:")
print(df_clean['casualty_severity'].value_counts())

# Create a binary labels for our target (severity levels)
df_clean['severity_levels'] = np.where(df_clean['casualty_severity'].isin([1, 2]),
print("\nCasualty severity groupped binary labels:")
print(df_clean['severity_levels'].value_counts())
```

```
Casualty severity data distribution:
casualty_severity
3    3249
2    1135
1      61
Name: count, dtype: int64

Casualty severity groupped binary labels:
severity_levels
0    3249
1    1196
Name: count, dtype: int64
```

3.5. Metadata

Variable	Type	Description	Notes
severity_levels	Binary	Determine pedestrian injury severity levels: 1 = fatal or serious injury, 0 = slight injury	Target variable
casualty_severity	Categorical	Recorded casualty severity levels: 1 = fatal, 2 = serious, 3 = slight	Used to create `severity_levels` variable, will be excluded from modelling
sex_of_casualty	Categorical	Sex of the pedestrian casualty	Input Predictor
age_of_casualty	Numeric	Age of the pedestrian casualty in years	Input Predictor
pedestrian_location	Categorical	Pedestrian's location relative to	Input Predictor

Variable	Type	Description	Notes
		the carriageway or crossing at collision	
pedestrian_movement	Categorical	Reported movement of the pedestrian at collision	Input Predictor
pedestrian_road_maintenance_worker	Categorical	Whether the pedestrian casualty was recorded as a road maintenance worker	Input Predictor
casualty_imd_decile	Categorical	Index of Multiple Deprivation decile associated with the area of casualty	Input Predictor
speed_limit	Categorical	Speed limit of the collision location	Input Predictor
first_road_class	Categorical	Classification of the first road at the collision location	Input Predictor
road_type	Categorical	Type of road where the collision occurred	Input Predictor
junction_detail	Categorical	Type of junction at the collision location	Input Predictor
junction_control	Categorical	Tyoe of traffic control present at the junction	Input Predictor
pedestrian_crossing	Categorical	Type of pedestrian	Input Predictor

Variable	Type	Description	Notes
		crossing facility present	
light_conditions	Categorical	Lighting conditions at collision	Input Predictor
weather_conditions	Categorical	Weather conditions at collision	Input Predictor
road_surface_conditions	Categorical	Condition of the road surface at the time of collision	Input Predictor
urban_or_rural_area	Categorical	Classification of the collision location as urban or rural	Input Predictor
trunk_road_flag	Categorical	Indicator of if the collision occurred on a trunk road	Input Predictor
vehicle_type	Categorical	Type of vehicle associated with the pedestrian casualty	Input Predictor
vehicle_manoeuvre	Categorical	Manoeuvre of the associated vehicle at collision	Input Predictor
junction_location	Categorical	Position of the associated vehicle relative to the junction	Input Predictor
dist_to_greenspace_m	Numeric	Distance to the nearest greenspace access point in metres	Spatial Input Predictor

Variable	Type	Description	Notes
time_period	Categorical	Time of day as Morning Peak, Daytime, Evening Peak, Evening and Night	Input Predictor
day_of_week	Categorical	Day of week on which the collision occurred	Input Predictor
day_type	Categorical	Weekday or weekend	Input Predictor

3.6. Exploratory Analysis

- ***3.6.1.*** The final casualty-level dataset contained 1,196, or 26.9%, fatal or serious and 3,249, or 73.1%, slight pedestrian casualties, highlighting class imbalance.
- ***3.6.2.*** Greenspace distance had a weak but significant positive correlation with speed limit, suggesting it may partly reflect surrounding road environment.
- ***3.6.3.*** Across common urban speed limits of 20–40mph, severe share was highest among older pedestrians and consistently highest on 40mph roads.
- ***3.6.4.*** Fatal/serious injury shares also varied spatially, but patterns should be interpreted cautiously where casualty counts are lower.

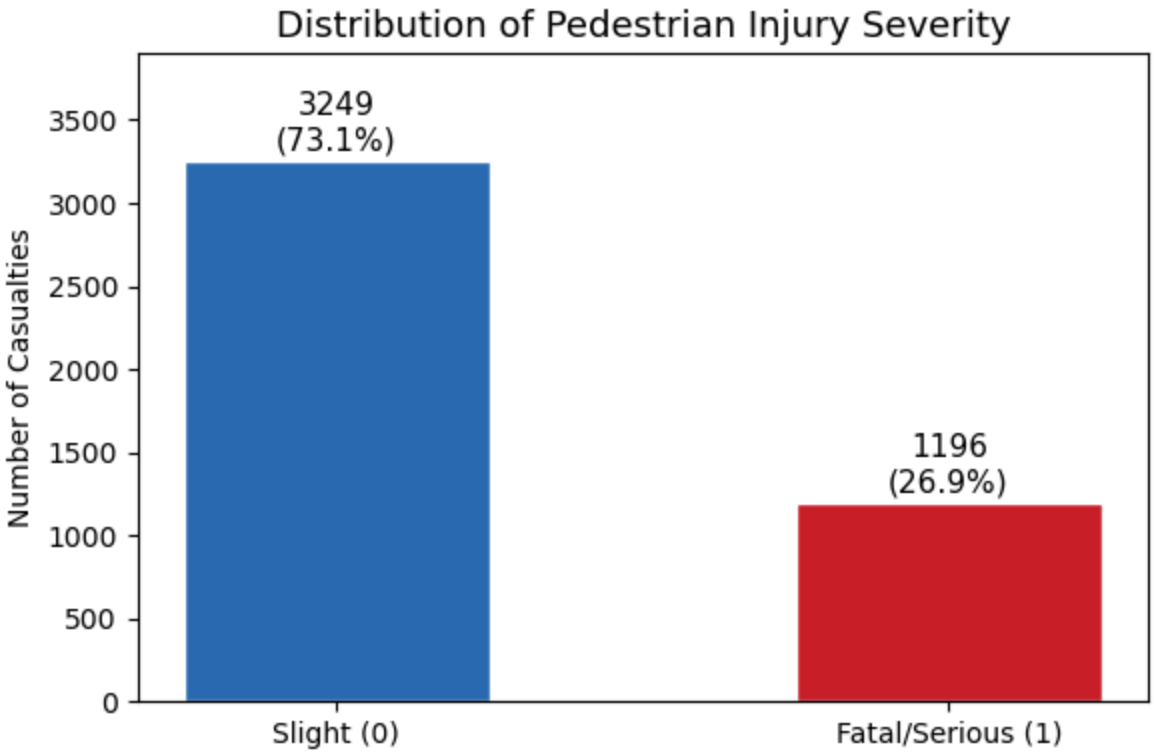
3.6.1. Pedestrian Severity Levels Data Distribution

In [22]:
```
severity_counts = df_clean['severity_levels'].value_counts()
labels = ['Slight (0)', 'Fatal/Serious (1)']
colors = ['#2B6CB0', '#C81E2A']

fig, ax = plt.subplots(figsize=(6, 4))
bars = ax.bar(labels, severity_counts.values, color=colors, edgecolor='white', widt

for bar, count in zip(bars, severity_counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 30,
            f'{count}\n({count/len(df_clean)*100:.1f}%)',
            ha='center', va='bottom', fontsize=11)

ax.set_title('Distribution of Pedestrian Injury Severity', fontsize=13)
ax.set_ylabel('Number of Casualties')
ax.set_ylim(0, severity_counts.max() * 1.2)
plt.tight_layout()
plt.show()
```



3.6.2. Speed Limit and Greenspace Distance Correlation Inspection

In [23]:
```
# Check correlation between greenspace distance and speed limit
greenspace_speedlimit_check = df_clean[['dist_to_greenspace_m', 'speed_limit']].cop
greenspace_speedlimit_check['speed_limit'] = pd.to_numeric(
    greenspace_speedlimit_check['speed_limit'],
    errors='coerce'
)
greenspace_speedlimit_check = greenspace_speedlimit_check.dropna()

rho, pval = spearmanr(
    greenspace_speedlimit_check['dist_to_greenspace_m'],
    greenspace_speedlimit_check['speed_limit']
)

print("Spearman correlation between greenspace distance and speed limit:")
print(f"ρ = {rho:.4f}, p < 0.001" if pval < 0.001 else f"ρ = {rho:.4f}, p = {pval:.

# Mean greenspace distance by speed limit
mean_greenspace = (
    df_clean.groupby('speed_limit', dropna=False)['dist_to_greenspace_m']
    .mean()
    .reset_index()
)

mean_greenspace['speed_limit'] = pd.to_numeric(mean_greenspace['speed_limit'], erro
mean_greenspace = mean_greenspace.dropna().sort_values('speed_limit')

print("\nMean greenspace distance by speed limit:")
print(mean_greenspace.to_string(index=False))
```

```
Spearman correlation between greenspace distance and speed limit:
ρ = 0.1235, p < 0.001

Mean greenspace distance by speed limit:
 speed_limit  dist_to_greenspace_m
          20            150.938074
          30            180.468014
          40            194.532716
          50            199.818479
          60            333.308242
          70            643.755792
```

In [24]:

```python
fig, ax = plt.subplots(figsize=(8, 5))

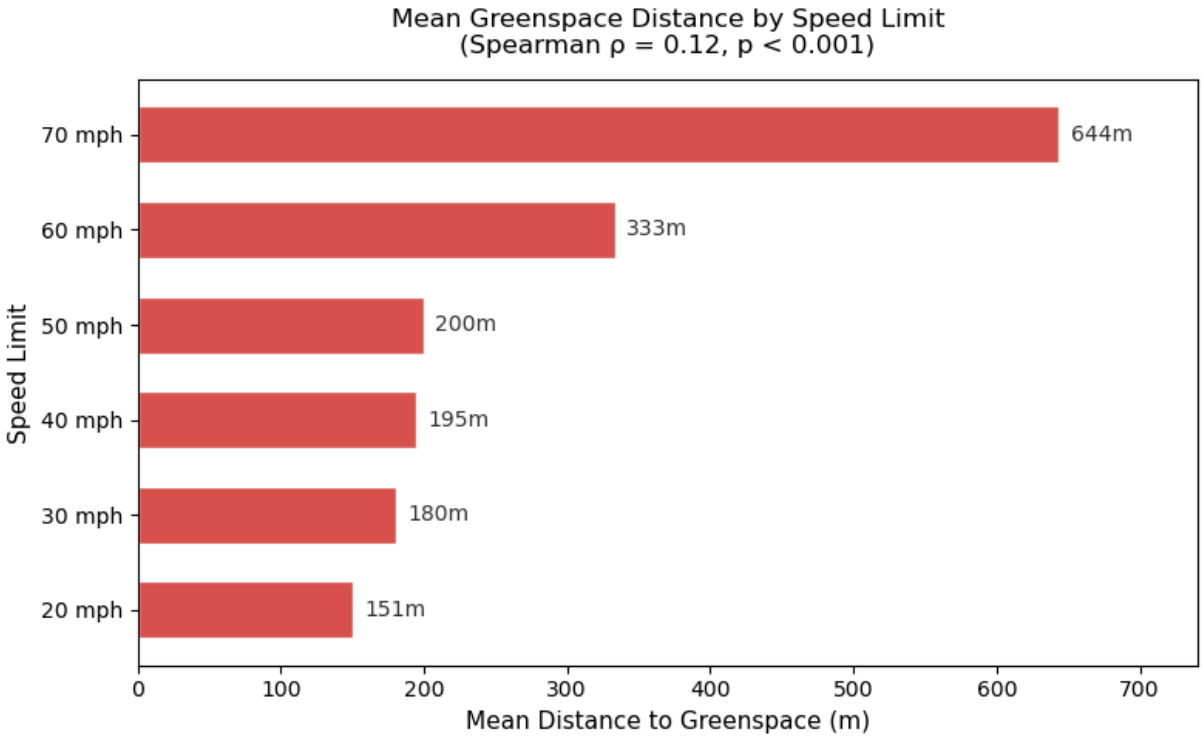
bars = ax.barh(
    mean_greenspace['speed_limit'].astype(int).astype(str) + " mph",
    mean_greenspace['dist_to_greenspace_m'],
    color="#d9534f",
    edgecolor="white",
    height=0.6
)

# Add value labels
for bar, val in zip(bars, mean_greenspace['dist_to_greenspace_m']):
    ax.text(
        val + 8,
        bar.get_y() + bar.get_height() / 2,
        f"{val:.0f}m",
        va="center",
        fontsize=10,
        color="#333333"
    )

p_text = "p < 0.001" if pval < 0.001 else f"p = {pval:.3f}"

ax.set_xlabel("Mean Distance to Greenspace (m)", fontsize=11)
ax.set_ylabel("Speed Limit", fontsize=11)
ax.set_title(
    "Mean Greenspace Distance by Speed Limit\n"
    f"(Spearman ρ = {rho:.2f}, {p_text})",
    fontsize=12,
    pad=12
)
ax.set_xlim(0, mean_greenspace['dist_to_greenspace_m'].max() * 1.15)

plt.tight_layout()
plt.show()
```



3.6.3. Age and Speed Limit on Fatal/ Serious Pedestrian Injury

In [25]:

```
# speed_limit remains categorical (object) in df_clean for modelling
# Create a temporary numeric version of speed_limit for heatmap visualisation only
df_clean['speed_limit'] = pd.to_numeric(df_clean['speed_limit'], errors='coerce')

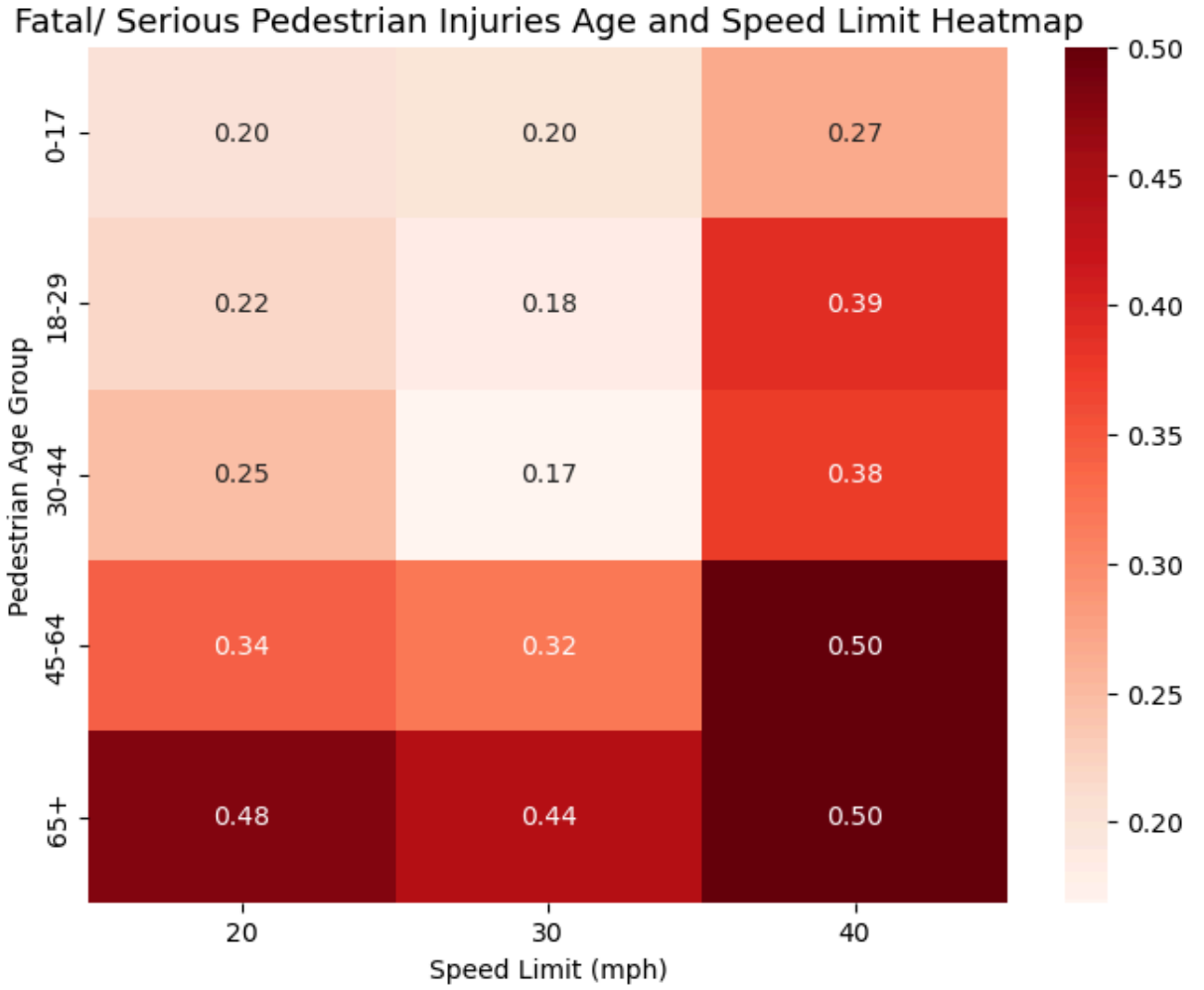
# Select low speed limit
# As 50, 60 and 70mph roads are for dual carriageways and motorways in the UK (http
target_speeds = [20, 30, 40]
df_filtered = df_clean[df_clean['speed_limit'].isin(target_speeds)].copy()

# Create Age Groups
age_bins = [0, 18, 30, 45, 65, 100]
age_labels = ['0-17', '18-29', '30-44', '45-64', '65+']
df_filtered['age_group'] = pd.cut(
    df_filtered['age_of_casualty'],
    bins=age_bins,
    labels=age_labels,
    right=False
)

# Pivot data table
pivot_table = df_filtered.pivot_table(
    index='age_group',
    columns='speed_limit',
    values='severity_levels',
    aggfunc='mean'
)

# Plot
plt.figure(figsize=(8, 6))
sns.heatmap(pivot_table, annot=True, cmap='Reds', fmt=".2f")
```

```
plt.title("Fatal/ Serious Pedestrian Injuries Age and Speed Limit Heatmap", fontsiz
plt.ylabel("Pedestrian Age Group")
plt.xlabel("Speed Limit (mph)")
plt.show()
```



3.6.4. Share of Pedestrian Casualties that were Fatal/ Serious by London MSOA

In [26]:
```
# Choropleth map: fatal/serious injury share among pedestrian casualties by MSOA

# Aggregate pedestrian casualty severity by MSOA
msoa_severity = (
    df_clean.groupby("MSOA11CD")["severity_levels"]
    .agg(
        total_casualties="count",
        fatal_serious="sum"
    )
    .reset_index()
)

# Calculate severe injury share
msoa_severity["severe_share"] = (
    msoa_severity["fatal_serious"] / msoa_severity["total_casualties"]
)
```

```
# Merge severity counts to MSOA boundaries
msoa_map = london_msoa.merge(
    msoa_severity,
    on="MSOA11CD",
    how="left"
)

# Fill missing count values
msoa_map["total_casualties"] = msoa_map["total_casualties"].fillna(0)
msoa_map["fatal_serious"] = msoa_map["fatal_serious"].fillna(0)

# Recalculate severe share safely
msoa_map["severe_share"] = np.where(
    msoa_map["total_casualties"] > 0,
    msoa_map["fatal_serious"] / msoa_map["total_casualties"],
    np.nan
)

# Only map MSOAs with at least 5 pedestrian casualties
msoa_map["severe_share_min5"] = np.where(
    msoa_map["total_casualties"] >= 5,
    msoa_map["severe_share"],
    np.nan
)

print("Summary Statistics:")
print(f"Total London MSOAs: {len(msoa_map)}")
print(f"MSOAs with at least 1 pedestrian casualty: {(msoa_map['total_casualties'] >
print(f"MSOAs shown on map, at least 5 casualties: {(msoa_map['total_casualties'] >
print(f"MSOAs with no pedestrian casualties recorded: {(msoa_map['total_casualties'

# Create category
msoa_map["map_status"] = np.select(
    [
        msoa_map["total_casualties"] == 0,
        (msoa_map["total_casualties"] > 0) & (msoa_map["total_casualties"] < 5),
        msoa_map["total_casualties"] >= 5
    ],
    [
        "No pedestrian casualties recorded",
        "Fewer than 5 pedestrian casualties",
        "Mapped severe-share value"
    ],
    default="Unknown"
)

# Plot
fig, ax = plt.subplots(figsize=(12, 10))

# No casualties
msoa_map[msoa_map["map_status"] == "No pedestrian casualties recorded"].plot(
    color="#f2f2f2",
    edgecolor="white",
    linewidth=0.2,
    ax=ax
)
```

```
# Fewer than 5 casualties
msoa_map[msoa_map["map_status"] == "Fewer than 5 pedestrian casualties"].plot(
    color="#d9d9d9",
    edgecolor="white",
    linewidth=0.2,
    ax=ax
)

# Map values
msoa_map[msoa_map["map_status"] == "Mapped severe-share value"].plot(
    column="severe_share_min5",
    cmap="OrRd",
    linewidth=0.2,
    edgecolor="white",
    legend=True,
    scheme="user_defined",
    classification_kwds={
        "bins": [0.25, 0.50, 0.75, 1.00]
    },
    ax=ax,
    legend_kwds={
        "loc": "lower right",
        "title": "Fatal/Serious Share"
    }
)

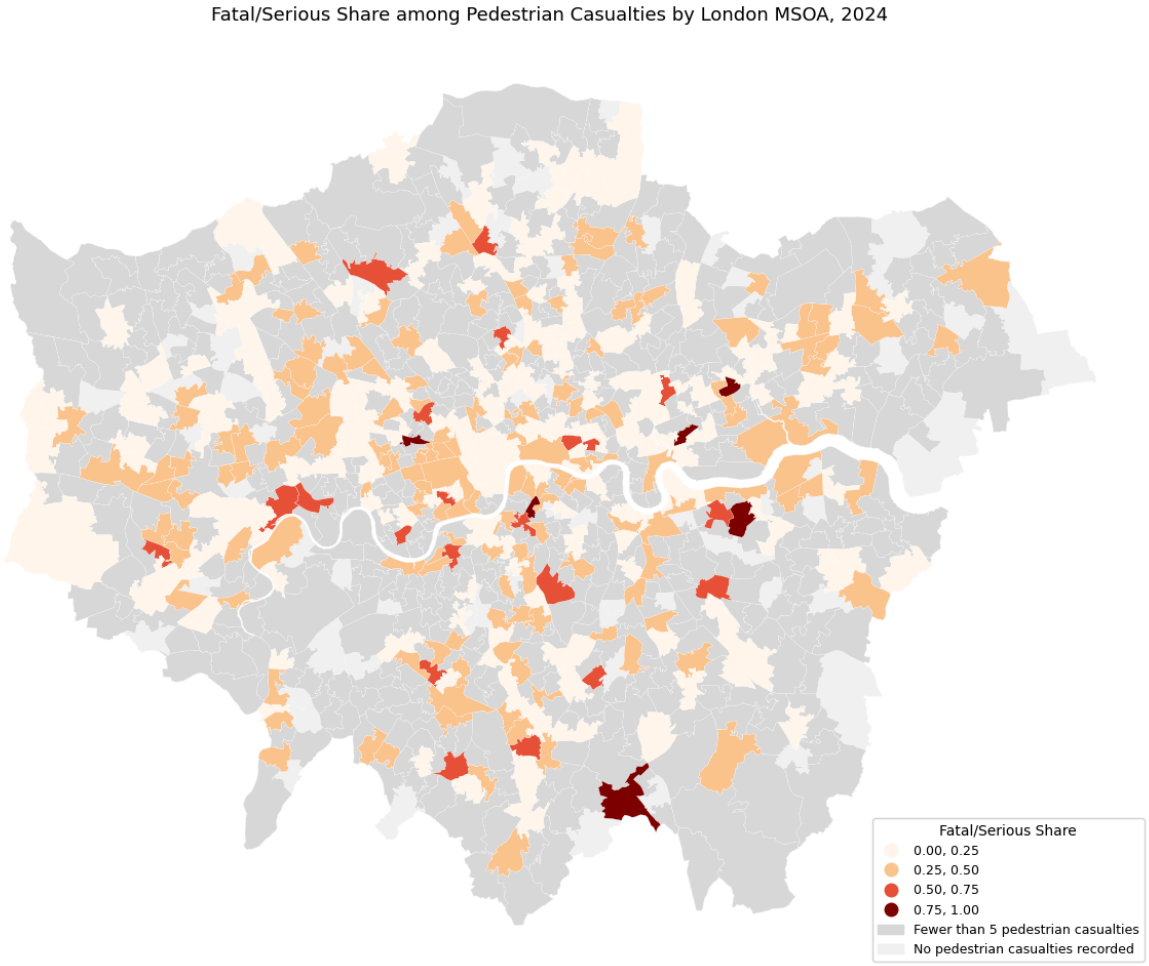
# Add grey legend categories
grey_patches = [
    mpatches.Patch(color="#d9d9d9", label="Fewer than 5 pedestrian casualties"),
    mpatches.Patch(color="#f2f2f2", label="No pedestrian casualties recorded")
]

legend = ax.get_legend()
if legend is not None:
    handles = legend.legend_handles + grey_patches
    labels = [text.get_text() for text in legend.get_texts()] + [
        "Fewer than 5 pedestrian casualties",
        "No pedestrian casualties recorded"
    ]
    legend.remove()
    ax.legend(
        handles=handles,
        labels=labels,
        loc="lower right",
        title="Fatal/Serious Share",
        fontsize=9,
        title_fontsize=10,
        frameon=True
    )

ax.set_title(
    "Fatal/Serious Share among Pedestrian Casualties by London MSOA, 2024",
    fontsize=13,
    pad=15
)
```

```
ax.set_axis_off()
plt.tight_layout()
plt.show()
```

```
Summary Statistics:
Total London MSOAs: 983
MSOAs with at least 1 pedestrian casualty: 919
MSOAs shown on map, at least 5 casualties: 379
MSOAs with no pedestrian casualties recorded: 64
```



4. Methodology

[go back to the top]

4.1. Justification of Machine Learning Modelling Approach

Machine learning approaches were used because road safety data are often noisy and it offers greater flexibility and require minimal prior assumptions (Dong et al., 2022; Tao et al., 2022). Logistic Regression provided an interpretable baseline for testing greenspace's incremental value, while Random Forest captured non-linear relationships, interactions and variable importance.

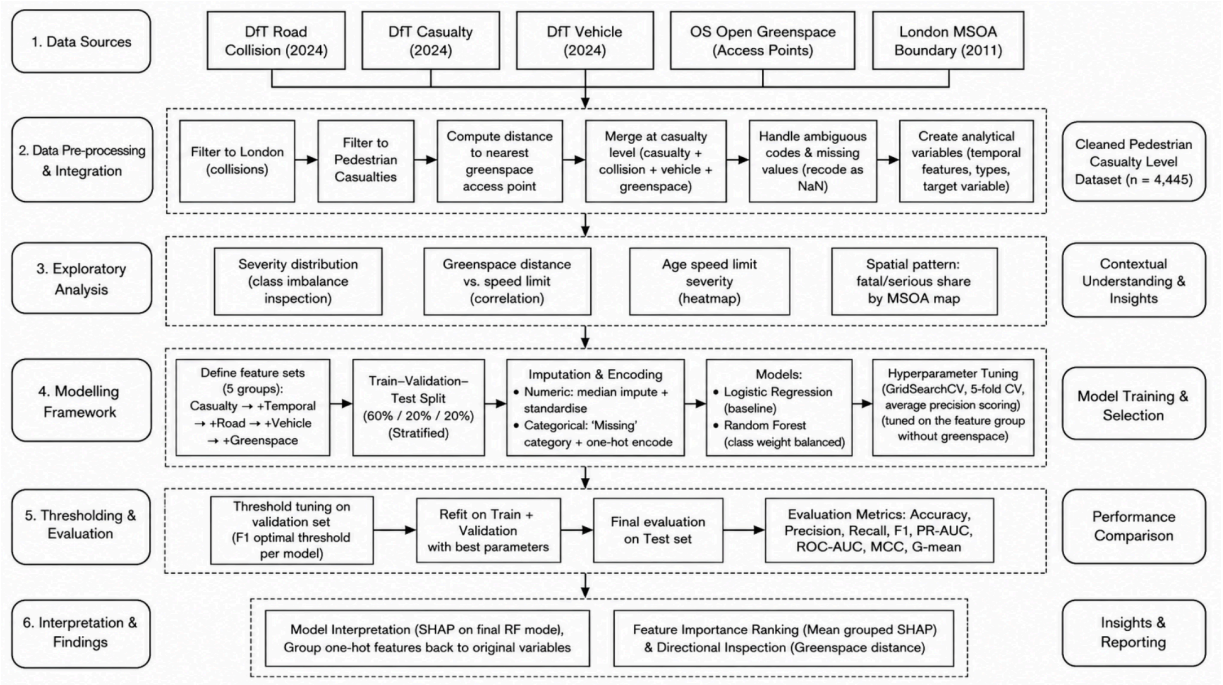
- **Logistic Regression (LR)**: $$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1X_1 + \ldots + \beta_nX_n)}}$$ An interpretable linear baseline for binary injury severity classification for testing greenspace's incremental value (Usman, Fu and Miranda-Moreno, 2016).
- **Random Forest (RF)**: $$\hat{y} = \text{mode}\{h_1(X), h_2(X), \ldots, h_T(X)\}$$ To capture non-linear relationships, interactions and variable importance.

4.2. Class Imbalance

As fatal/serious casualties comprised only 26.9% of observations, models used balanced class weights. Evaluation prioritised recall, F1-score, PR-AUC, MCC and G-mean rather than accuracy, given the higher cost of misclassifying severe injuries and class imbalance.

4.3. Analytical Framework in Modelling

In [27]:
```
# To display methodology flowchart for pdf submission
from IPython.display import Image, display
display(Image(filename="flowchart.png"))
```



4.3.1. Imputation Pipeline

Pre-processing pipeline implemented where numeric variables were imputed using the median and standardised, while categorical variables were imputed with a "Missing" category and one-hot encoded using the ColumnTransformer.

In [28]:
```
# Define Imputation Pipeline
def impute_preprocess(numeric_f, categorical_f):

    # Numeric pipeline
```

```
    # Impute NaNs with median and standardise
    num_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler',  StandardScaler())
    ])

    # Categorical pipeline
    # Impute missing with a 'Missing' category and one-hot encode
    cat_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='constant', fill_value='Missing')),
        ('onehot',  OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ])

    # Apply the pipelines to its corresponding column types
    return ColumnTransformer([
        ('num', num_pipeline, numeric_f),
        ('cat', cat_pipeline, categorical_f)
    ])
```

4.3.2. Hyperparameter Tuning and Model Refit

$$AP = \sum_{n}(R_n - R_{n-1})P_n$$

RF hyperparameters were tuned using GridSearchCV with stratified five-fold cross-validation and `average_precision` as the scoring metric to reflect performance on the minority class in training (scikit-learn.org., n.d.). Tuning was conducted on feature group casualty, temporal, road, and vehicle variables, excluding greenspace, allowing evaluation of features' incremental contribution. The optimal hyperparameters were then refitted across all feature group specifications to ensure comparability, such that performance differences reflect predictor inclusion than model specification. Probability thresholds were optimised on validation set using F1-score for each model, and final performance was evaluated on testing.

4.3.3. Performance Metrics Evaluation

Model performance focused using recall, F1-score, PR-AUC, MCC, and G-mean. Including MCC as it balances sensitivity, specificity across four confusion matrix categories and G-means by capturing the geometric mean of recall across classes, providing a more reliable and balanced measure of classification performance (Chicco and Jurman, 2020; Ri and Kim, 2020; Sujon et al., 2025). These metrics were prioritised over accuracy due to class imbalance, ensuring balanced evaluation across both severity classes while emphasising correct identification of fatal/serious outcomes.

$$ Matthews Correlation Coefficient (MCC) = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} $$

$$ G\text{-}mean = \sqrt{\frac{TP}{TP + FN} \cdot \frac{TN}{TN + FP}} $$

In [29]:
```python
# Define G-means
def gmean_score(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0

    return np.sqrt(sensitivity * specificity)
```

4.3.4. Model Interpretation

While ML approaches can establish complex, non-linear relationships between variables and injury severity outcomes, they also face limitations of black-box nature (Dong et al., 2022). Full RF model was interpreted using SHAP to assess relative contribution of greenspace proximity within the complete feature set, not as casual evidence. One-hot encoded features were grouped to their original variables for readable global feature importance rankings. They indicate relative predictive contribution, while directional SHAP used only to inspect greenspace proximity patterns.

5. Results and Discussion

[go back to the top]

5.1. Train Test Split

Data used stratification to preserve proportion of fatal/serious and slight casualties across splits. Models firstly trained on 60% training, with classification thresholds tuned on 20% validation set using F1-score. Final models then refitted on the combined training and validation data and evaluated on 20% test set, ensuring no testing information influenced training/ threshold selections.

In [30]:
```python
# Define input variables and target variable
X = df_clean[numeric_features + categorical_features].copy()
y = df_clean['severity_levels'].copy()

# 60% train, 20% validation, 20% test
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y,
    # Testing 20% and training + validation 80%
    test_size= 0.20,
    # For reproducibility
    random_state= 42,
    # Ensure Fatal/ Serioous and Fatal have the same proportion of split
    stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval,
    # Take 25% of the 80% for validation, which is 20% of total
```

```
    test_size= 0.25,
    random_state= 42,
    stratify=y_trainval
)

# Confirm amount of data for train validatation and testing before modelling
print(f"Train: {X_train.shape[0]:,} rows ({X_train.shape[0]/len(X)*100:.0f}%)")
print(f"Validation: {X_val.shape[0]:,} rows ({X_val.shape[0]/len(X)*100:.0f}%)")
print(f"Test: {X_test.shape[0]:,} rows ({X_test.shape[0]/len(X)*100:.0f}%)")
```

```
Train: 2,667 rows (60%)
Validation: 889 rows (20%)
Test: 889 rows (20%)
```

In [31]:
```
# Define feature selection groups as dictionaries for later modelling

feature_groups = {
    # What can casualty factors alone predict?
    'Casualty': {
        'numeric':     casualty_numeric,
        'categorical': casualty_categorical
    },
    # What does adding temporal add on top of that?
    'Casualty + Temporal': {
        'numeric':     casualty_numeric,
        'categorical': casualty_categorical + temporal_categorical
    },
    # What does adding road add on top of that?
    'Casualty + Temporal + Road': {
        'numeric':     casualty_numeric,
        'categorical': casualty_categorical + temporal_categorical + road_categoric
    },
    # What does adding vehicle add on top of that
    'Casualty + Temporal + Road + Vehicle': {
        'numeric':     casualty_numeric,
        'categorical': casualty_categorical + temporal_categorical
                       + road_categorical + vehicle_categorical
    },
    # What does greenspace add on top of everything?
    'Casualty + Temporal + Road + Vehicle + Greenspace': {
        'numeric':     casualty_numeric + greenspace_numeric,
        'categorical': casualty_categorical + temporal_categorical
                       + road_categorical + vehicle_categorical
    }
}
```

5.2. Logistic Regression

Class weighted LR found adding greenspace distance produced minor change in PR-AUC, from 0.425 to 0.427, while F1, G-mean, MCC slightly decreased. This suggests little linear incremental value after accounting for casualty, temporal, road and vehicle variables.

In [32]:
```
# Logistic Regression Baseline Model (with and without greenspace proximity)
```

```python
# Define two logistic regression feature sets
logit_feature_sets = {
    'Logistic Regression: Without Greenspace': {
        'numeric': casualty_numeric,
        'categorical': casualty_categorical + temporal_categorical + road_categoric
    },
    'Logistic Regression: With Greenspace': {
        'numeric': casualty_numeric + greenspace_numeric,
        'categorical': casualty_categorical + temporal_categorical + road_categoric
    }
}

logit_results = {}
logit_models = {}

for model_name, feats in logit_feature_sets.items():

    num_f = feats['numeric']
    cat_f = feats['categorical']
    all_f = num_f + cat_f

    # Build logistic regression pipeline
    logit_pipe = Pipeline([
        ('pre', impute_preprocess(num_f, cat_f)),
        ('clf', LogisticRegression(
            class_weight='balanced',
            max_iter=1000,
            solver='lbfgs'
        ))
    ])

    # Fit model on training data
    logit_pipe.fit(X_train[all_f], y_train)

    # Predict probabilities on validation set for threshold tuning
    val_probs = logit_pipe.predict_proba(X_val[all_f])[:, 1]

    # Tune classification threshold using validation F1
    threshold_rows = []
    for th in np.arange(0.10, 0.91, 0.05):
        val_preds = (val_probs >= th).astype(int)
        threshold_rows.append({
            'threshold': round(th, 2),
            'precision': precision_score(y_val, val_preds, zero_division=0),
            'recall': recall_score(y_val, val_preds, zero_division=0),
            'f1': f1_score(y_val, val_preds, zero_division=0)
        })

    threshold_df = pd.DataFrame(threshold_rows)
    best_threshold = threshold_df.sort_values('f1', ascending=False).iloc[0]['thres

    # Refit model on train + validation data
    logit_pipe.fit(X_trainval[all_f], y_trainval)

    # Evaluate on test data
    test_probs = logit_pipe.predict_proba(X_test[all_f])[:, 1]
```

```
    test_preds = (test_probs >= best_threshold).astype(int)

    logit_results[model_name] = {
        'Threshold': round(best_threshold, 2),
        'Accuracy': round(accuracy_score(y_test, test_preds), 4),
        'Precision': round(precision_score(y_test, test_preds, zero_division=0), 4)
        'Recall': round(recall_score(y_test, test_preds, zero_division=0), 4),
        'F1': round(f1_score(y_test, test_preds, zero_division=0), 4),
        'G_mean': round(gmean_score(y_test, test_preds), 4),
        'MCC': round(matthews_corrcoef(y_test, test_preds), 4),
        'ROC_AUC': round(roc_auc_score(y_test, test_probs), 4),
        'PR_AUC': round(average_precision_score(y_test, test_probs), 4)
    }

    logit_models[model_name] = {
        'model': logit_pipe,
        'num': num_f,
        'cat': cat_f,
        'all': all_f,
        'threshold': best_threshold,
        'test_probs': test_probs,
        'test_preds': test_preds
    }

    print(f"\n{model_name}")
    print(f"Selected threshold: {best_threshold:.2f}")
    print(classification_report(
        y_test,
        test_preds,
        target_names=['Slight', 'Fatal/Serious']
    ))

# Display logistic regression comparison
logit_results_df = pd.DataFrame(logit_results).T
print("\nLogistic Regression Baseline Model Comparison:")
print(logit_results_df[['Accuracy', 'Recall', 'F1', 'G_mean', 'MCC', 'PR_AUC']].rou
```

```
Logistic Regression: Without Greenspace
Selected threshold: 0.45
               precision    recall  f1-score   support

       Slight       0.85      0.52      0.65       650
Fatal/Serious       0.37      0.74      0.49       239

     accuracy                           0.58       889
    macro avg       0.61      0.63      0.57       889
 weighted avg       0.72      0.58      0.61       889


Logistic Regression: With Greenspace
Selected threshold: 0.45
               precision    recall  f1-score   support

       Slight       0.85      0.52      0.65       650
Fatal/Serious       0.36      0.74      0.49       239

     accuracy                           0.58       889
    macro avg       0.61      0.63      0.57       889
 weighted avg       0.72      0.58      0.60       889


Logistic Regression Baseline Model Comparison:
                                        Accuracy  Recall     F1  G_mean  \
Logistic Regression: Without Greenspace     0.584   0.745  0.490   0.625
Logistic Regression: With Greenspace        0.582   0.745  0.489   0.623

                                          MCC  PR_AUC
Logistic Regression: Without Greenspace  0.240   0.425
Logistic Regression: With Greenspace     0.237   0.427
```

5.3. Random Forest

RF then used to test non-linearity, where performance improved most when road and vehicle variables added. The best model Casualty + Temporal + Road + Vehicle, found 0.504 F1, 0.279 MCC. Adding greenspace reduced F1 to 0.481, MCC to 0.222, with limited PR-AUC changes, suggesting limited incremental predictive value.

In [33]:
```python
# Find best params using feature group: Casualty + Temporal + Road + Vehicle

# Reference group for tuning
tune_group = feature_groups['Casualty + Temporal + Road + Vehicle']
tune_num = tune_group['numeric']
tune_cat = tune_group['categorical']
tune_all = tune_num + tune_cat

rf_tune_pipe = Pipeline([
    ('pre', impute_preprocess(tune_num, tune_cat)),
    ('clf', RandomForestClassifier(
        random_state=42,
        n_jobs=-1
    ))
```

```
])

param_grid_rf = {
    'clf__n_estimators': [200, 300],          # Reduce variance
    'clf__max_depth': [5, 10, None],          # To balance between underfitting and
    'clf__min_samples_leaf': [2, 5, 10],      # Reduce overfitting risks in minority
    'clf__max_features': ['sqrt', 'log2'],    # Controls randomness
    'clf__class_weight': ['balanced']         # Address class imbalance problems
}

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

grid_rf = GridSearchCV(
    rf_tune_pipe,
    param_grid_rf,
    scoring='average_precision',   # PR-AUC
    cv=cv,
    n_jobs=-1,
    verbose=1
)

grid_rf.fit(X_train[tune_all], y_train)

best_rf_params = {
    k.replace('clf__', ''): v
    for k, v in grid_rf.best_params_.items()
}

print("Best RF parameters from reference model:")
print(best_rf_params)
print(f"Best CV PR-AUC: {grid_rf.best_score_:.4f}")
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits
Best RF parameters from reference model:
{'class_weight': 'balanced', 'max_depth': None, 'max_features': 'sqrt', 'min_samples
_leaf': 5, 'n_estimators': 200}
Best CV PR-AUC: 0.4301
```

In [34]:

```
# Retrain model with best params found above but finds a different threshold for ea

group_results = {}
group_models = {}

threshold_grid = np.arange(0.10, 0.91, 0.05)

for group_name, feats in feature_groups.items():

    num_f = feats['numeric']
    cat_f = feats['categorical']
    all_f = num_f + cat_f

    pipe = Pipeline([
```

```
        ('pre', impute_preprocess(num_f, cat_f)),
        ('clf', RandomForestClassifier(
            **best_rf_params,
            random_state=42,
            n_jobs=-1
        ))
    ])

    # Fit on training set only
    pipe.fit(X_train[all_f], y_train)

    # Tune threshold on validation set
    val_probs = pipe.predict_proba(X_val[all_f])[:, 1]

    threshold_rows = []
    for th in threshold_grid:
        val_preds = (val_probs >= th).astype(int)
        threshold_rows.append({
            'threshold': th,
            'precision': precision_score(y_val, val_preds, zero_division=0),
            'recall': recall_score(y_val, val_preds, zero_division=0),
            'f1': f1_score(y_val, val_preds, zero_division=0)
        })

    threshold_df = pd.DataFrame(threshold_rows)
    best_threshold = threshold_df.sort_values('f1', ascending=False).iloc[0]['thres

    # Refit on train + validation
    pipe.fit(X_trainval[all_f], y_trainval)

    # Evaluate once on test
    test_probs = pipe.predict_proba(X_test[all_f])[:, 1]
    test_preds = (test_probs >= best_threshold).astype(int)

    group_results[group_name] = {
        'Threshold': round(best_threshold, 2),
        'Accuracy': round(accuracy_score(y_test, test_preds), 4),
        'Precision': round(precision_score(y_test, test_preds, zero_division=0), 4)
        'Recall': round(recall_score(y_test, test_preds), 4),
        'F1': round(f1_score(y_test, test_preds), 4),
        'G_mean': round(gmean_score(y_test, test_preds), 4),
        'MCC': round(matthews_corrcoef(y_test, test_preds), 4),
        'ROC_AUC': round(roc_auc_score(y_test, test_probs), 4),
        'PR_AUC': round(average_precision_score(y_test, test_probs), 4)
}

    group_models[group_name] = {
        'model': pipe,
        'features': all_f,
        'numeric': num_f,
        'categorical': cat_f,
        'threshold': best_threshold,
        'test_probs': test_probs,
        'test_preds': test_preds
    }
```

```
    print(f"\nRandom Forest: {group_name}")
    print(f"Selected threshold: {best_threshold:.2f}")
    print(classification_report(
        y_test,
        test_preds,
        target_names=['Slight', 'Fatal/Serious']
    ))
```

```
Random Forest: Casualty
Selected threshold: 0.35
              precision    recall  f1-score   support

       Slight       0.85      0.27      0.41       650
Fatal/Serious       0.30      0.87      0.45       239

     accuracy                           0.43       889
    macro avg       0.57      0.57      0.43       889
 weighted avg       0.70      0.43      0.42       889


Random Forest: Casualty + Temporal
Selected threshold: 0.35
              precision    recall  f1-score   support

       Slight       0.84      0.29      0.43       650
Fatal/Serious       0.30      0.85      0.45       239

     accuracy                           0.44       889
    macro avg       0.57      0.57      0.44       889
 weighted avg       0.69      0.44      0.43       889


Random Forest: Casualty + Temporal + Road
Selected threshold: 0.40
              precision    recall  f1-score   support

       Slight       0.85      0.50      0.63       650
Fatal/Serious       0.36      0.77      0.49       239

     accuracy                           0.57       889
    macro avg       0.61      0.64      0.56       889
 weighted avg       0.72      0.57      0.60       889


Random Forest: Casualty + Temporal + Road + Vehicle
Selected threshold: 0.45
              precision    recall  f1-score   support

       Slight       0.83      0.67      0.75       650
Fatal/Serious       0.42      0.64      0.50       239

     accuracy                           0.66       889
    macro avg       0.63      0.65      0.62       889
 weighted avg       0.72      0.66      0.68       889


Random Forest: Casualty + Temporal + Road + Vehicle + Greenspace
Selected threshold: 0.40
              precision    recall  f1-score   support

       Slight       0.84      0.52      0.64       650
Fatal/Serious       0.36      0.73      0.48       239

     accuracy                           0.57       889
```

```
   macro avg       0.60      0.62      0.56       889
weighted avg       0.71      0.57      0.60       889
```

In [35]:
```python
# Random Forest results
rf_results_df = pd.DataFrame(group_results).T
print("\nRandom Forest Model Comparison:")
print(rf_results_df[['Accuracy', 'Recall', 'F1', 'G_mean', 'MCC', 'PR_AUC']].round(
```

```
Random Forest Model Comparison:
                                                Accuracy  Recall     F1  \
Casualty                                           0.430   0.866  0.450
Casualty + Temporal                                0.439   0.845  0.447
Casualty + Temporal + Road                         0.575   0.766  0.492
Casualty + Temporal + Road + Vehicle               0.664   0.636  0.504
Casualty + Temporal + Road + Vehicle + Greenspace  0.575   0.732  0.481

                                                G_mean    MCC  PR_AUC
Casualty                                         0.483  0.142   0.376
Casualty + Temporal                              0.494  0.137   0.382
Casualty + Temporal + Road                       0.622  0.242   0.431
Casualty + Temporal + Road + Vehicle             0.655  0.279   0.430
Casualty + Temporal + Road + Vehicle + Greenspace  0.615  0.222   0.428
```

5.4. Model Comparison

In [36]:
```python
# Model Comparisons
results_cols = ['Accuracy', 'Recall', 'F1', 'G_mean', 'MCC', 'PR_AUC']
combined_results = {}

for name, res in logit_results.items():
    combined_results[name] = {k: v for k, v in res.items() if k != 'Threshold'}

for name, res in group_results.items():
    combined_results[f'Random Forest: {name}'] = {
        k: v for k, v in res.items() if k != 'Threshold'
    }

results_df = pd.DataFrame(combined_results).T

print("\nModel Comparison:")
print(results_df[results_cols].round(3))
```

```
Model Comparison:
                                                Accuracy  Recall     F1  \
Logistic Regression: Without Greenspace            0.584   0.745  0.490
Logistic Regression: With Greenspace               0.582   0.745  0.489
Random Forest: Casualty                            0.430   0.866  0.450
Random Forest: Casualty + Temporal                 0.439   0.845  0.447
Random Forest: Casualty + Temporal + Road          0.575   0.766  0.492
Random Forest: Casualty + Temporal + Road + Veh...  0.664   0.636  0.504
Random Forest: Casualty + Temporal + Road + Veh...  0.575   0.732  0.481

                                                G_mean    MCC  PR_AUC
Logistic Regression: Without Greenspace          0.625  0.240   0.425
Logistic Regression: With Greenspace             0.623  0.237   0.427
Random Forest: Casualty                          0.483  0.142   0.376
Random Forest: Casualty + Temporal               0.494  0.137   0.382
Random Forest: Casualty + Temporal + Road        0.622  0.242   0.431
Random Forest: Casualty + Temporal + Road + Veh...  0.655  0.279   0.430
Random Forest: Casualty + Temporal + Road + Veh...  0.615  0.222   0.428
```

In [37]:

```python
# Precision-Recall Curves
fig, ax = plt.subplots(figsize=(10, 6))

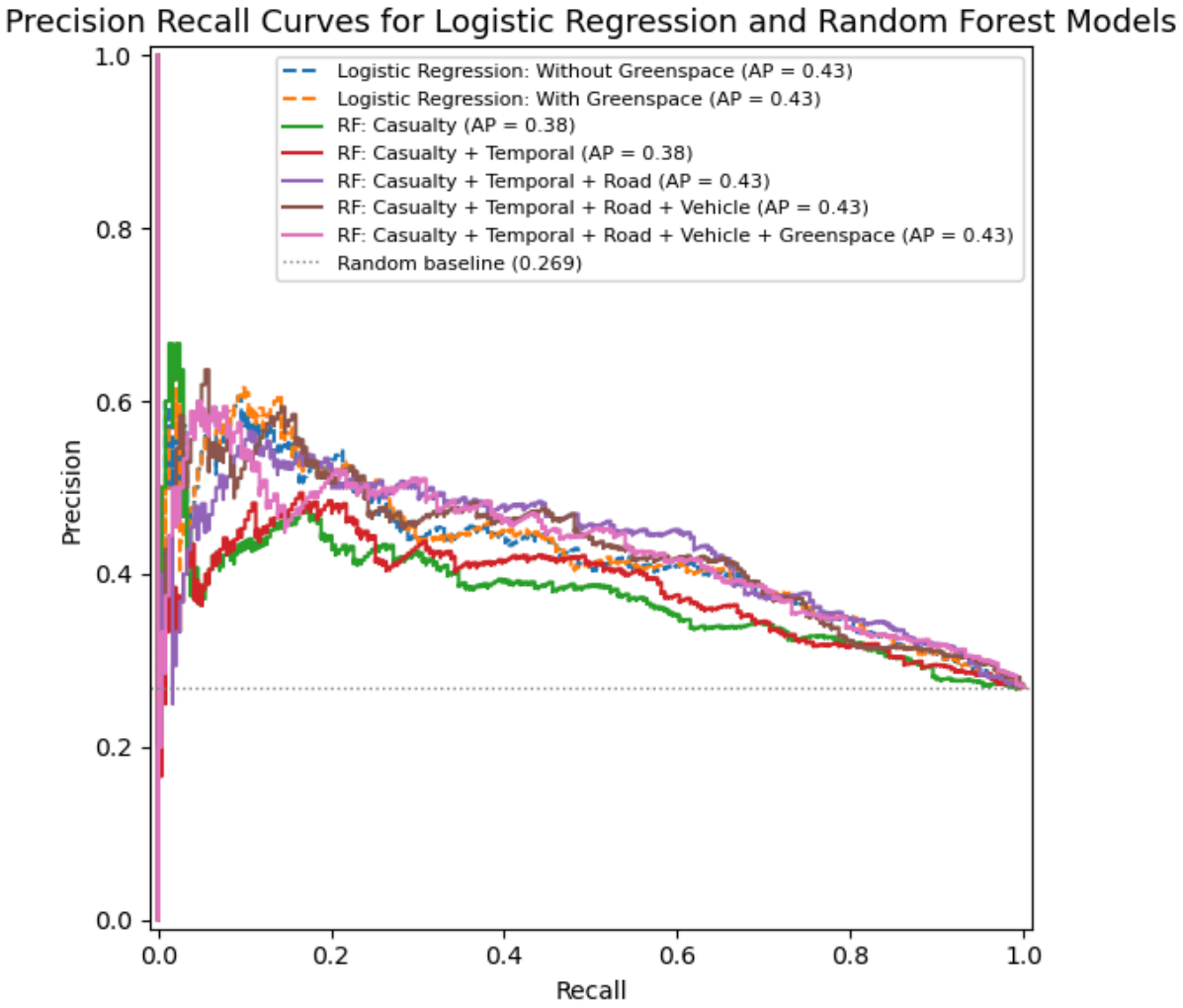
# Logistic regression models
for name, model_info in logit_models.items():
    PrecisionRecallDisplay.from_predictions(
        y_test,
        model_info['test_probs'],
        name=name,
        ax=ax,
        linestyle='--'
    )

# Random forest feature-group models
for name, model_info in group_models.items():
    PrecisionRecallDisplay.from_predictions(
        y_test,
        model_info['test_probs'],
        name=f'RF: {name}',
        ax=ax
    )

# Random baseline = prevalence of fatal/serious class
ax.axhline(
    y=y_test.mean(),
    color='grey',
    linestyle=':',
    linewidth=1,
    label=f'Random baseline ({y_test.mean():.3f})'
)

ax.set_title(
    'Precision Recall Curves for Logistic Regression and Random Forest Models',
    fontsize=13
)
ax.set_xlabel('Recall')
ax.set_ylabel('Precision')
```

```
ax.legend(fontsize=8, loc='upper right')
plt.tight_layout()
plt.show()
```



5.5. SHAP Model Interpretation

In [38]:
```
# SHAP interpretation for the already-fitted full Random Forest model

full_group_name = 'Casualty + Temporal + Road + Vehicle + Greenspace'

full_group = group_models[full_group_name]

pipe_shap = full_group['model']       # already fitted pipeline
full_all = full_group['features']
full_num = full_group['numeric']
full_cat = full_group['categorical']

# Extract fitted preprocessing and Random Forest model
pre_fitted = pipe_shap.named_steps['pre']
rf_fitted = pipe_shap.named_steps['clf']

# Sample test data for SHAP
X_shap_sample = X_test[full_all].sample(
```

```
    n=min(500, len(X_test)),
    random_state=42
)

# Apply the already-fitted preprocessing
X_transformed = pre_fitted.transform(X_shap_sample)

if hasattr(X_transformed, 'toarray'):
    X_transformed = X_transformed.toarray()

X_transformed = np.asarray(X_transformed, dtype=np.float64)

# Get feature names after preprocessing
raw_names = pre_fitted.get_feature_names_out()
clean_names = [
    name.replace('num__', '').replace('cat__', '')
    for name in raw_names
]

# Compute SHAP values
explainer = shap.TreeExplainer(rf_fitted)
shap_values = explainer(X_transformed)

# Keep class 1 = Fatal/Serious
if len(shap_values.values.shape) == 3:
    shap_class1_values = shap_values.values[:, :, 1]
else:
    shap_class1_values = shap_values.values

# Group one-hot encoded columns back to original variables
all_original = full_num + full_cat
feat_names = list(clean_names)

grouped_shap = np.zeros((shap_class1_values.shape[0], len(all_original)))

for col_idx, orig in enumerate(all_original):
    if orig in full_num:
        if orig in feat_names:
            i = feat_names.index(orig)
            grouped_shap[:, col_idx] = shap_class1_values[:, i]
    else:
        indices = [
            i for i, name in enumerate(feat_names)
            if name.startswith(orig + '_') or name == orig
        ]
        if indices:
            grouped_shap[:, col_idx] = shap_class1_values[:, indices].sum(axis=1)


# Rank variables by mean absolute SHAP value
# Group one-hot encoded SHAP values back to original variables through aggregating
all_original = full_num + full_cat
feat_names = list(clean_names)

grouped_abs_shap = np.zeros((shap_class1_values.shape[0], len(all_original)))
```

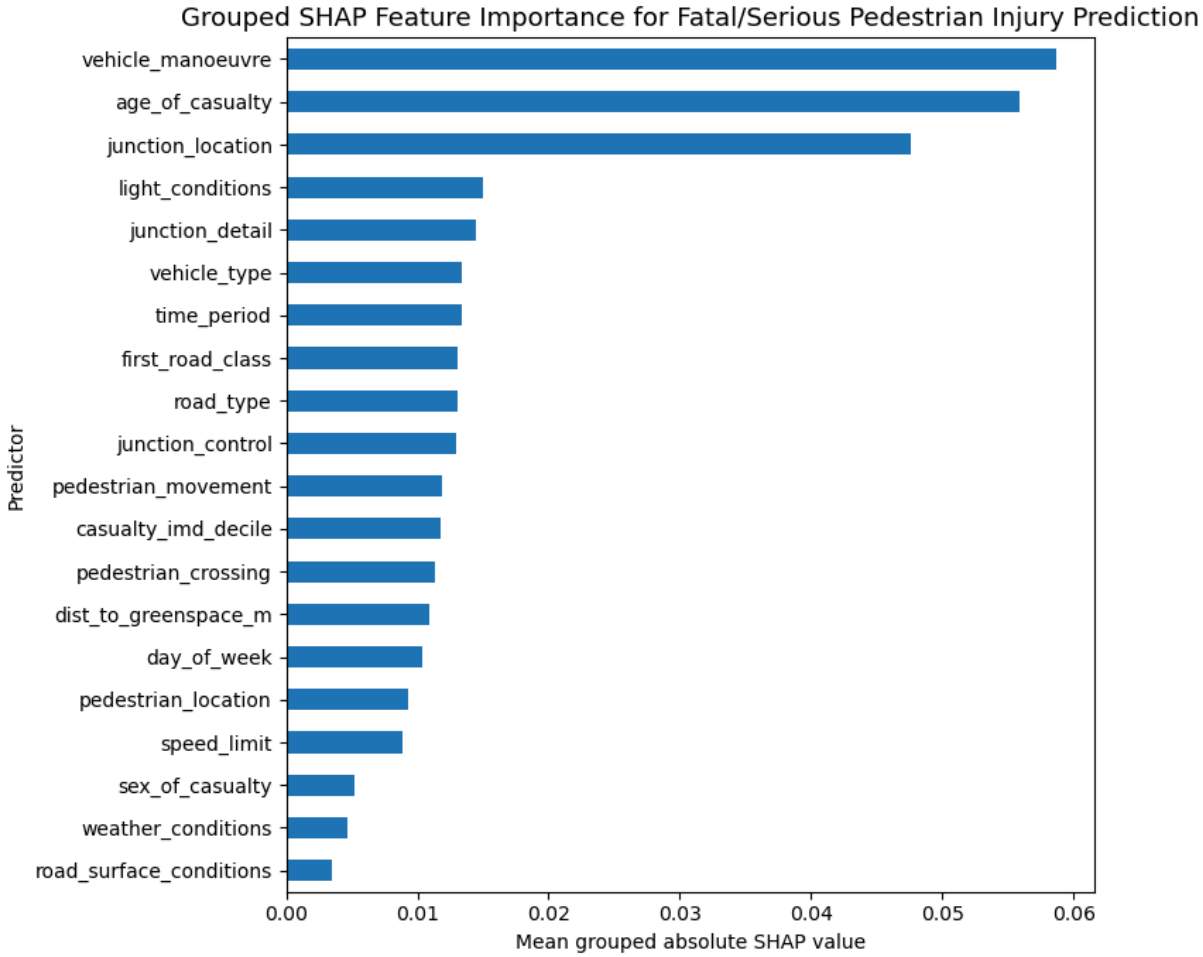
```
for col_idx, orig in enumerate(all_original):
    if orig in full_num:
        if orig in feat_names:
            i = feat_names.index(orig)
            grouped_abs_shap[:, col_idx] = np.abs(shap_class1_values[:, i])
    else:
        indices = [
            i for i, name in enumerate(feat_names)
            if name.startswith(orig + '_') or name == orig
        ]
        if indices:
            grouped_abs_shap[:, col_idx] = np.abs(shap_class1_values[:, indices]).s

# Rank variables by mean grouped absolute SHAP value
shap_ranked = pd.Series(
    grouped_abs_shap.mean(axis=0),
    index=all_original
).sort_values(ascending=True)
```

In [39]:

```
# Plot grouped SHAP importance
fig, ax = plt.subplots(figsize=(8, 7))
shap_ranked.tail(20).plot(kind='barh', ax=ax)

ax.set_title(
    'Grouped SHAP Feature Importance for Fatal/Serious Pedestrian Injury Prediction
    fontsize=13
)
ax.set_xlabel('Mean grouped absolute SHAP value')
ax.set_ylabel('Predictor')
plt.tight_layout()
plt.show()
```



In [40]:
```
# SHAP ranking
shap_ranked_desc = shap_ranked.sort_values(ascending=False)

print("Features ranked by mean grouped absolute SHAP value:")
print(shap_ranked_desc.round(5).to_string())

gs_rank = list(shap_ranked_desc.index).index('dist_to_greenspace_m') + 1

print(f"\nGreenspace grouped mean absolute SHAP: {shap_ranked_desc['dist_to_greensp
print(f"Greenspace rank: {gs_rank} of {len(shap_ranked_desc)}")
```

```
Features ranked by mean grouped absolute SHAP value:
vehicle_manoeuvre                       0.05864
age_of_casualty                         0.05586
junction_location                       0.04761
light_conditions                        0.01503
junction_detail                         0.01443
vehicle_type                            0.01341
time_period                             0.01340
first_road_class                        0.01308
road_type                               0.01303
junction_control                        0.01299
pedestrian_movement                     0.01190
casualty_imd_decile                     0.01175
pedestrian_crossing                     0.01140
dist_to_greenspace_m                    0.01091
day_of_week                             0.01040
pedestrian_location                     0.00927
speed_limit                             0.00884
sex_of_casualty                         0.00522
weather_conditions                      0.00468
road_surface_conditions                 0.00349
day_type                                0.00271
pedestrian_road_maintenance_worker      0.00056
urban_or_rural_area                     0.00050
trunk_road_flag                         0.00002

Greenspace grouped mean absolute SHAP: 0.01091
Greenspace rank: 14 of 24
```

In [41]:

```python
# Directional SHAP inspection for greenspace distance using raw metres
greenspace_idx = feat_names.index('dist_to_greenspace_m')

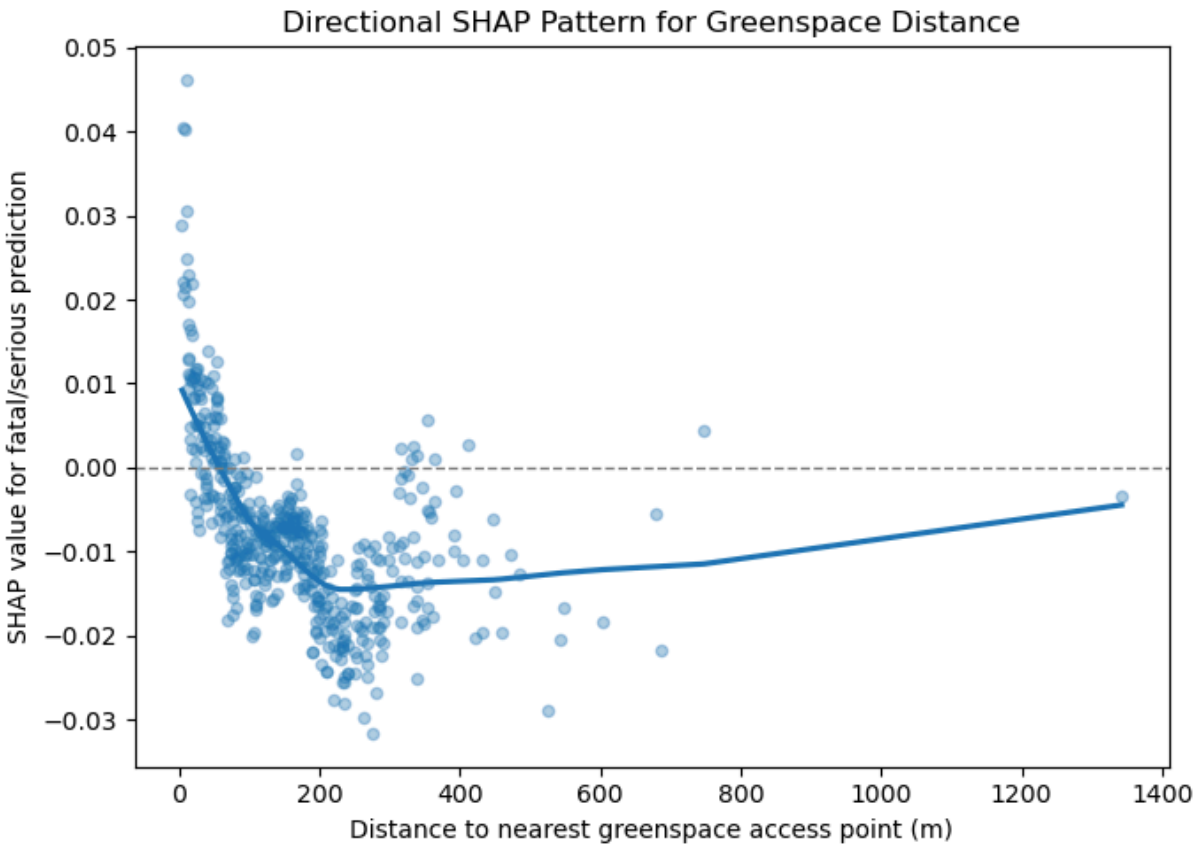
greenspace_shap_df = pd.DataFrame({
    'dist_to_greenspace_m': X_shap_sample['dist_to_greenspace_m'].values,
    'shap_value': shap_class1_values[:, greenspace_idx]
}).dropna()

fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(
    greenspace_shap_df['dist_to_greenspace_m'],
    greenspace_shap_df['shap_value'],
    alpha=0.35,
    s=20
)

# Add a smoothed trend line
sns.regplot(
    data=greenspace_shap_df,
    x='dist_to_greenspace_m',
    y='shap_value',
    scatter=False,
    lowess=True,
    ax=ax
)

ax.axhline(0, linestyle='--', linewidth=1, color='grey')
```

```
ax.set_title('Directional SHAP Pattern for Greenspace Distance')
ax.set_xlabel('Distance to nearest greenspace access point (m)')
ax.set_ylabel('SHAP value for fatal/serious prediction')
plt.tight_layout()
plt.show()
```



5.6. Summary of Findings

Injury severity most strongly predicted by road and subsequently vehicle factors, with RF indicating weak greenspace incremental value. Temporal variables contributed comparatively little, while best model included casualty, temporal, road and vehicle predictors.

RQ1: Casualty age, vehicle manoeuvre and junction-related characteristics were the most important predictors of fatal or serious pedestrian injury, highlighting the role of pedestrian vulnerability, vehicle behaviour and the road environment in shaping severity outcomes.

RQ2: Distance to greenspace access points did not meaningfully improve under non-linear modelling. While SHAP analysis suggests some contextual variation with greater greenspace distance, its contribution is comparably relatively weak with higher predicted severity and key predictors. This likely reflects a limitation of the modelling framework, which conditions on recorded casualties and does not account for pedestrian exposure. This does not imply that greenspace is irrelevant to pedestrian safety, greenspace proximity may influence where collisions occur, but may not translate directly into severity outcomes.

Key limitations include potential reporting inconsistencies, where data predominantly relied on STATS19 reported police records. Focusing on a single year sample 2024 only, supports temporal consistency but may limit sample size. Lastly, model validation used random cross-validation splits rather than considering spatial partitioning, which may limit geographic generalisability.

6. Conclusion

[go back to the top]

To conclude, pedestrian injury severity in London is better predicted by casualty, vehicle, road environment characteristics than broader spatial context like greenspace proximity. This suggests that policy strategies targeting junction design, vehicle manoeuvres, lighting are more directly relevant to injury severity. While urban greening remains important for walking and public health, its role in influencing severity appears less direct and mediated through patterns of pedestrian activity. Furthermore, absence of pedestrian exposure data makes it difficult to determine whether higher casualty numbers reflect greater injury risk or simply higher pedestrian presence. Therefore, future research should consider integrating pedestrian flow, more comprehensive greenspace measures such as size, connectivity.

7. References

[go back to the top]

Al Hasanat, H.A. et al. (2025) 'Interpretable machine learning for imbalanced pedestrian injury severity prediction in urban Jordan', Journal of King Saud University – Engineering Sciences, 37(7), p. 39. Available at: https://doi.org/10.1007/s44444-025-00053-3.

Chicco, D. and Jurman, G. (2020) 'The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation', BMC Genomics, 21. Available at: https://doi.org/10.1186/s12864-019-6413-7.

Clifton, K.J., Burnier, C.V. and Akar, G. (2009) 'Severity of injury resulting from pedestrian–vehicle crashes: What can we learn from examining the built environment?', Transportation Research Part D: Transport and Environment, 14(6), pp. 425–436. Available at: https://doi.org/10.1016/j.trd.2009.01.001.

Dong, S. et al. (2022) Predicting and Analyzing Road Traffic Injury Severity Using Boosting-Based Ensemble Learning Models with SHAPley Additive exPlanations - PMC. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8910532/ (Accessed: 14 April 2026).

Elalouf, A., Birfir, S. and Rosenbloom, T. (2023) 'Developing machine-learning-based models to diminish the severity of injuries sustained by pedestrians in road traffic incidents', Heliyon, 9(11), p. e21371. Available at: https://doi.org/10.1016/j.heliyon.2023.e21371.

Hossain, A. et al. (2026) 'Leveraging Explainable Machine Learning Techniques to Estimate Crash Severity for Different Pedestrian Actions', Transportation Research Record, p. 03611981261420130. Available at: https://doi.org/10.1177/03611981261420130.

Malin, F., Silla, A. and Mladenović, M.N. (2020) 'Prevalence and factors associated with pedestrian fatalities and serious injuries: case Finland', European Transport Research Review, 12(1), p. 29. Available at: https://doi.org/10.1186/s12544-020-00411-z.

Mayor of London (2021) The London Plan 2021. Available at: https://www.london.gov.uk/programmes-strategies/planning/london-plan/the-london-plan-2021-table-contents (Accessed: 14 April 2026).

Mayor of London and Transport for London (n.d.) Walking Action Plan. Available at: https://content.tfl.gov.uk/mts-walking-action-plan.pdf (Accessed: 14 April 2026).

Pour-Rouholamin, M. and Zhou, H. (2016) 'Investigating the risk factors associated with pedestrian injury severity in Illinois', Journal of Safety Research, 57, pp. 9–17. Available at: https://doi.org/10.1016/j.jsr.2016.03.004.

Ri, J. and Kim, H. (2020) 'G-mean based extreme learning machine for imbalance learning', Digital Signal Processing, 98, p. 102637. Available at: https://doi.org/10.1016/j.dsp.2019.102637.

scikit-learn.org. (n.d.). sklearn.metrics.average_precision_score — scikit-learn 0.24.1 documentation. [online] Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html.

Sujon, K.M. et al. (2025) 'Accuracy, precision, recall, f1-score, or MCC? empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models', Journal of Big Data, 12(1), p. 268. Available at: https://doi.org/10.1186/s40537-025-01313-4.

Tao, W. et al. (2022) 'An Advanced Machine Learning Approach to Predicting Pedestrian Fatality Caused by Road Crashes: A Step toward Sustainable Pedestrian Safety', Sustainability, 14(4), p. 2436. Available at: https://doi.org/10.3390/su14042436.

Transport for London (n.d.) Vision Zero for London, Vision Zero for London. Available at: https://www.tfl.gov.uk/corporate/protecting-our-network/road-safety/vision-zero-for-london (Accessed: 14 April 2026).

Usman, T., Fu, L. and Miranda-Moreno, L.F. (2016) 'Injury severity analysis: comparison of multilevel logistic regression models and effects of collision data aggregation', Journal of Modern Transportation, 24(1), pp. 73–87. Available at: https://doi.org/10.1007/s40534-016-0096-4.

World Health Organization (2023) Road traffic injuries. Available at: https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries (Accessed: 14 April

2026).